PRACTICAL-1

Part A — Practice Programs
Work these in your lab-NN/ folder before the project: do the first one or two in the lab.

1. Vending machine (VendingMachine.java, main in this file)
(a) Define an enum Coin with constants ONE, TWO, FIVE, TEN.
(b) In main, set a snack price of 15 and a running total of 0; create a Scanner.
(c) Loop: read a coin name, use a switch expression to convert the Coin to its value (ONE→1, TWO→2, FIVE→5, TEN→10), add it to the total, and print the total so far.
(d) Stop the loop once the total reaches 15 or more.
(e) Print the change to return (total − 15).
Expected output: TEN then FIVE prints “Paid. Change: 0”; TEN then TEN prints “Paid. Change: 5”.

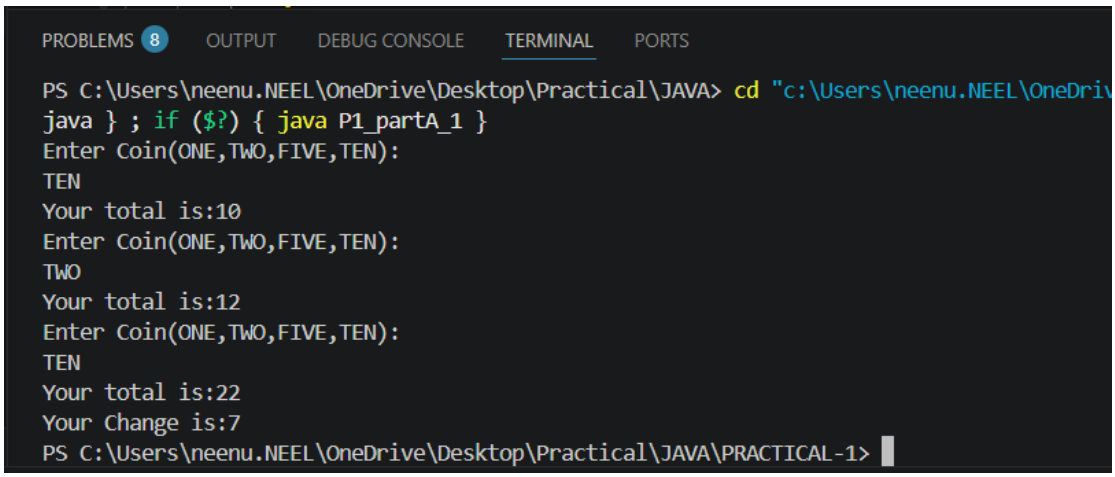
CODE:

```
import java.util.*;
public class P1_partA_1 {
   enum Coin{
      ONE,
      TWO,
      FIVE,
      TEN
   }
   public static void main(String []args){
      int snackprice=15;
      int Total=0;
      Scanner sc=new Scanner(System.in);

      while(Total<snackprice){
         System.out.println("Enter Coin(ONE,TWO,FIVE,TEN):");
         Coin coin=Coin.valueOf(sc.next().toUpperCase());

         int value=switch(coin){
            case ONE -> 1;
            case TWO -> 2;
            case FIVE -> 5;
            case TEN -> 10;
         };

         Total=Total+value;
         System.out.println("Your total is:" + Total);
      }
      System.out.println("Your Change is:"+(Total-snackprice));
      sc.close();
   }
}
```



2. Toll booth (TollBooth.java, main in this file)
(a) Define a record Vehicle(String number, String type).
(b) In main, keep a running toll total and three counters (bike, car, truck).
(c) Loop until the user types “done” for the number: build a Vehicle, use a switch expression on its type for the toll (bike→20, car→50, truck→150), add to the total, and increment that type’s counter.
(d) After the loop, print the total toll and the type with the highest count.
Expected output: two cars + one bike → “Total toll: 120”, “Most frequent: car”.

CODE:

```
import java.util.*;

record Vehicle(String number,String type){}

public class P1_partA_2{
   public static void main(String[] args){
      Scanner sc=new Scanner(System.in);

      int total=0;
      int bike=0;
      int car=0;
      int truck=0;

      while(true){
         System.out.print("Enter vehicle number (done to stop):");
         String number=sc.next();

         if(number.equalsIgnoreCase("done")){
            break;
         }

         System.out.print("Enter vehicle type (bike/car/truck):");
         String type=sc.next().toLowerCase();

         Vehicle v=new Vehicle(number,type);

         int toll=switch(type){
            case "bike"->20;
            case "car"->50;
            case "truck"->150;
            default->0;
```

```
        };

        total+=toll;

        if(type.equals("bike")){
           bike++;
        }else if(type.equals("car")){
           car++;
        }else if(type.equals("truck")){
           truck++;
        }
     }

     System.out.println("Total toll: "+ total);

     if(bike>car&&bike>truck)
        {
        System.out.println("Most frequent: bike");
        }
     else if(car>bike&&car>truck)
        {
        System.out.println("Most frequent: car");
        }
     else
        {
        System.out.println("Most frequent: truck");
        }
     sc.close();
  }
}
```

```
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-1> cd "c:\User
 P1_partA_2.java } ; if ($?) { java P1_partA_2 }
Enter vehicle number (done to stop):2
Enter vehicle type (bike/car/truck):bike
Enter vehicle number (done to stop):1
Enter vehicle type (bike/car/truck):car
Enter vehicle number (done to stop):done
Total toll: 70
Most frequent: truck
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-1>
```

Part B — Project Work (Hour 2): MiniBank

MiniBank is the console banking application you will build across the whole semester. In this first lab you set up your tools and create the project and its menu shell — the screen the user navigates. Every later lab attaches one real banking feature behind these menu options.

1. Set up your environment: open the project in GitHub Codespaces (or install JDK 17+ and an IDE locally), and create a private GitHub repository named oop-minibank-<rollno>; add the faculty as a collaborator.
2. Create a public class named MiniBank that contains the main method.
3. Define a record named BankInfo with two String fields, name and branch (a record is a short, read-only class; Java auto-generates its constructor and field accessors). Print one BankInfo object as the application header.
4. Define an enum named MenuOption with the constants OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER and EXIT — these are the actions MiniBank will offer.
5. Display the numbered menu to the user and read the chosen number with a Scanner.
6. Use a switch expression on the choice to print a short placeholder for that option (for example, “Deposit — to be implemented in a later lab”). Keep showing the menu in a loop until the user selects EXIT, then print a goodbye message.

25DCS093

CODE:

```
import java.util.*;
record BankInfo(String name,String branch){}

enum MenuOption{
   OPEN_ACCOUNT,
   DEPOSIT,
   WITHDRAW,
   TRANSFER,
   EXIT
}

public class P1_partB{
   public static void main(String[] args){

      Scanner sc=new Scanner(System.in);

      BankInfo bank=new BankInfo("Mini Bank","Anand");

      System.out.println("Bank Name: " + bank.name());
      System.out.println("Branch: "+ bank.branch());

      while(true){

         System.out.println("1.Open Account");
         System.out.println("2.Deposit");
         System.out.println("3.Withdraw");
         System.out.println("4.Transfer");
         System.out.println("5.Exit");

         System.out.print("Enter your choice: ");
         int choice=sc.nextInt();

         MenuOption option=switch(choice){
            case 1->MenuOption.OPEN_ACCOUNT;
            case 2->MenuOption.DEPOSIT;
            case 3->MenuOption.WITHDRAW;
            case 4->MenuOption.TRANSFER;
            case 5->MenuOption.EXIT;
            default->null;
         };

         if(option==null){
            System.out.println("Invalid Choice");
            continue;
         }

         switch(option){
            case OPEN_ACCOUNT->System.out.println("Open Account -To be implemented later.");
            case DEPOSIT->System.out.println("Deposit -To be implemented later.");
            case WITHDRAW->System.out.println("Withdraw -To be implemented later.");
            case TRANSFER->System.out.println("Transfer-To be implemented later.");
            case EXIT->{System.out.println("Thank You!!!!");
               sc.close();
               return;
            }
         }
      }
   }
```

}

```
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-1> cd "c:\Users\neenu.
 P1_partB.java } ; if ($?) { java P1_partB }
Bank Name: Mini Bank
Branch: Anand
1.Open Account
2.Deposit
3.Withdraw
4.Transfer
5.Exit
Enter your choice: 2
Deposit -To be implemented later.
1.Open Account
2.Deposit
3.Withdraw
4.Transfer
5.Exit
Enter your choice: 3
Withdraw -To be implemented later.
1.Open Account
2.Deposit
3.Withdraw
4.Transfer
5.Exit
Enter your choice: 4
Transfer-To be implemented later.
1.Open Account
2.Deposit
3.Withdraw
4.Transfer
5.Exit
Enter your choice: 5
Thank You!!!!
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-1>
```

KEY QUESTION:

1.What is the role of the JVM, and what file does the javac compiler produce?

=>The JVM runs Java bytecode, and the javac compiler converts the .java source file into a .class file containing bytecode.

2.**How does a switch expression differ from a traditional switch statement?**

=>A switch expression returns a value and can use yield, while a traditional switch statement performs actions without directly returning a value.

3.Why is an enum a good choice for a fixed set of menu options, and a record for read-only data?

=>An enum is good for a fixed set of menu options because it provides predefined constants, while a record is suitable for read-only data because it is compact and automatically provides final fields, constructors, and accessors.

PRACTICAL-2

Part A — Practice Programs

Work these in your lab-NN/ folder before the project: do the first one or two in the lab.

1. Smart thermostat (Thermostat.java, main in this file)
(a) Create class Thermostat with private String location, private int temperature, private static final int MIN=16 and MAX=30, and private static int activeCount=0.
(b) Constructor(location, startTemp): store location; set temperature to startTemp if it is within MIN..MAX else 22; increment activeCount.
(c) Constructor(location): chain with this(location, 22).
(d) raise(): if temperature < MAX add 1, else print "Already at maximum (30)".
(e) lower(): if temperature > MIN subtract 1, else print "Already at minimum (16)".
(f) Add getTemperature() (no setter) and static getActiveCount().
(g) In main: create two thermostats, call raise() 10 times then lower() 20 times in loops (printing the temperature each time), then print getActiveCount().

Expected output: temperature rises to 30 then "Already at maximum", falls to 16 then "Already at minimum", count 2.

CODE:

```
class Thermostat{

private String location;
private int temperature;
private static final int MIN=16;
private static final int MAX=30;
private static int activeCount=0;

  Thermostat(String location, int startTemp)
  {
    this.location=location;

    if(startTemp>=MIN && startTemp<=MAX)
    {
      temperature=startTemp;
    }
    else
    {
      temperature=22;
    }
    activeCount++;
  }

  Thermostat(String location)
  {
    this(location,22);
  }

  void raise(){
    if(temperature < MAX)
    {
      temperature++;
    }
    else
    {
      System.out.println("Already at maximum(30)");
    }
  }
```

```
    void lower(){
        if(temperature > MIN)
        {
            temperature--;
        }
        else
        {
            System.out.println("Already at minimum(16)");
        }
    }

    int getTemperature(){
        return temperature;
    }

    static int getActiveCount(){
        return activeCount;
    }
}


public class P2_partA_1{
    public static void main(String[] args){

        Thermostat t=new Thermostat("BEDROOOM",24);
        Thermostat t1=new Thermostat("LIVING_ROOM");

        System.out.println("Raising Temperature:");

        for(int i=1;i<=10;i++)
        {
            t.raise();
            System.out.println("Temperature: "+ t.getTemperature());
        }

        System.out.println("Lowering Temperature:");

        for(int i=1;i<=20;i++)
        {
            t.lower();
            System.out.println("Temperature: "+ t.getTemperature());
        }

        System.out.println("Active Thermostats: "+ Thermostat.getActiveCount());
    }
}
```

```
1 error
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-2> cd "c:\Users\neenu.NEEL\OneD
Raising Temperature:
Temperature: 25
Temperature: 26
Temperature: 27
Temperature: 28
Temperature: 29
Temperature: 30
Already at maximum(30)
Temperature: 30
Already at maximum(30)
Temperature: 30
Already at maximum(30)
Temperature: 30
Already at maximum(30)
Temperature: 30
Lowering Temperature:
Temperature: 29
Temperature: 28
Temperature: 27
Temperature: 26
Temperature: 25
Temperature: 24
Temperature: 23
Temperature: 22
Temperature: 21
Temperature: 20
Temperature: 19
Temperature: 18
Temperature: 17
Temperature: 16
Already at minimum(16)
Temperature: 16
Already at minimum(16)
Temperature: 16
Already at minimum(16)
Temperature: 16
Already at minimum(16)
Temperature: 16
Already at minimum(16)
Temperature: 16
Already at minimum(16)
Temperature: 16
Active Thermostats: 2
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-2>
```

2. Cinema show (CinemaShow.java, main in this file)
(a) Create class CinemaShow with private String title, private int seatsAvailable, private final int capacity, and private static int totalBooked=0.
(b) Constructor(title, capacity): set both and seatsAvailable = capacity.
Constructor(title): chain with this(title, 100).
(c) book(int n): if n ≤ seatsAvailable, reduce seatsAvailable by n, add n to totalBooked, return true; else return false unchanged.
(d) cancel(int n): increase seatsAvailable by n but never above capacity.
(e) Add getSeatsAvailable() and static getTotalBooked().
(f) In main: run a sequence of book and cancel calls, printing the result and seats after each, then print getTotalBooked().
Expected output: booking more than available prints false and leaves seats unchanged; totalBooked equals the successful bookings.

CODE:

```
class CinemaShow{

   private String title;
   private int seatsAvailable;
   private final int capacity;
   private static int totalBooked=0;

   CinemaShow(String title,int capacity){
      this.title=title;
      this.capacity=capacity;
      seatsAvailable=capacity;
   }

   CinemaShow(String title){
      this(title,100);
   }

   boolean book(int n){
      if(n<=seatsAvailable){
         seatsAvailable=seatsAvailable-n;
         totalBooked=totalBooked+n;
         return true;
      }
      else{
         return false;
      }
   }

   void cancel(int n){
      seatsAvailable=seatsAvailable+n;

      if(seatsAvailable>capacity){
         seatsAvailable=capacity;
      }
   }

   int getSeatsAvailable(){
      return seatsAvailable;
   }

   static int getTotalBooked(){
      return totalBooked;
   }
}

public class P2_partA_2{

   public static void main(String[] args){

      CinemaShow show=new CinemaShow("Avengers",50);

      System.out.println(show.book(20));
      System.out.println("Seats Available: "+show.getSeatsAvailable());

      System.out.println(show.book(15));
      System.out.println("Seats Available: "+show.getSeatsAvailable());

      show.cancel(10);
```

```
    System.out.println("Seats Available: "+show.getSeatsAvailable());

    System.out.println(show.book(30));
    System.out.println("Seats Available: "+show.getSeatsAvailable());

    System.out.println("Total Booked: "+CinemaShow.getTotalBooked());
  }
```

```
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-2> cd "c:\Users\nee
 P2_partA_2.java } ; if ($?) { java P2_partA_2 }
true
Seats Available: 30
true
Seats Available: 15
Seats Available: 25
false
Seats Available: 25
Total Booked: 35
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-2>
```

Part B — Project Work (Hour 2): MiniBank

In Practical 1 you built the empty menu shell. Now you create the data the bank works on — the Customer and Account classes. Using encapsulation, an account's balance can change only through deposit() and withdraw(), never directly. These two classes are the foundation every later lab extends.

1. Create a class named Customer with private fields name, email and mobile (all String) and a private final field customerId (String). Use a private static long field customerCounter and a private static method generateCustomerId() to create ids such as "CUST101". Add a constructor and public getter methods.
2. Create a class named Account with private fields: a final String accountNumber, a String ownerName, a long balance (whole rupees) and a boolean active. Generate accountNumber with a private static counter (for example "AC0001").
3. Add two constructors to Account: one taking ownerName and an opening balance; one taking only ownerName that calls the first using this(ownerName, 0).
4. Add a public method deposit(long amount) that adds amount to balance. Add a public method withdraw(long amount) that subtracts amount and returns true when the balance is sufficient, otherwise returns false without changing the balance. Add getter methods. Do NOT add a public setter for balance.
5. In main, create three Account objects inside an Account[] array, perform a few deposits and withdrawals, and print each balance.

CODE:

```
class Customer{
   private String name;
   private String email;
   private String mobile;
   private final String customerId;

   private static long customerCounter=101;

   private static String generateCustomerId(){
      return "CUST"+customerCounter++;
   }

   public Customer(String name,String email,String mobile){
      this.name=name;
      this.email=email;
      this.mobile=mobile;
      customerId=generateCustomerId();
   }

   public String getName(){
      return name;
   }

   public String getEmail(){
      return email;
   }

   public String getMobile(){
      return mobile;
   }

   public String getCustomerId(){
      return customerId;
   }
}

class Account{
   private final String accountNumber;
   private String ownerName;
   private long balance;
   private boolean active;

   private static int accountCounter=1;

   private static String generateAccountNumber(){
      return String.format("AC%04d",accountCounter++);
   }

   public Account(String ownerName,long balance){
      accountNumber=generateAccountNumber();
      this.ownerName=ownerName;
      this.balance=balance;
      active=true;
   }

   public Account(String ownerName){
      this(ownerName,0);
```

```
    }

    public void deposit(long amount){
        balance+=amount;
    }

    public boolean withdraw(long amount){
        if(balance>=amount){
            balance-=amount;
            return true;
        }
        return false;
    }

    public String getAccountNumber(){
        return accountNumber;
    }

    public String getOwnerName(){
        return ownerName;
    }

    public long getBalance(){
        return balance;
    }

    public boolean isActive(){
        return active;
    }
}

public class P2_partB{
    public static void main(String[]args){

        Customer c1=new Customer("Neel","neeupatel07@gmail.com","9726138766");
        Customer c2=new Customer("Selvy","selvymodi5@gmail.com","8866677336");

        Account[]accounts=new Account[3];

        accounts[0]=new Account(c1.getName(),1000);
        accounts[1]=new Account(c2.getName());
        accounts[2]=new Account("Kavy",500);

        accounts[0].deposit(25000);
        accounts[0].withdraw(100);

        accounts[1].deposit(3500);
        accounts[1].withdraw(150);

        accounts[2].deposit(200);
        accounts[2].withdraw(500);

        for(int i=0;i<accounts.length;i++){
            System.out.println("Account Number: "+accounts[i].getAccountNumber());
            System.out.println("Owner Name: "+accounts[i].getOwnerName());
            System.out.println("Balance: Rs."+accounts[i].getBalance());
            System.out.println();
        }
    }
}
```

```
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-2> cd "c:\Users\
 P2_partB.java } ; if ($?) { java P2_partB }
Account Number: AC0001
Owner Name: Neel
Balance: Rs.25900

Account Number: AC0002
Owner Name: Selvy
Balance: Rs.3350

Account Number: AC0003
Owner Name: Kavy
Balance: Rs.200

PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-2>
```

KEY QUESTION:

1.What is encapsulation, and how do private fields with public getters enforce it?
=>Encapsulation means hiding data inside a class; private fields prevent direct access, while public getters provide controlled access to read their values.

2.Why is accountNumber declared final?
=>accountNumber is declared final so its value cannot be changed after it is assigned, ensuring every account keeps a unique and fixed account number.

3.What is the difference between a static field and an instance field when generating ids?
=>A static field is shared by all objects and is useful for generating unique IDs, while an instance field belongs to each object separately and cannot track a common ID count.

PRACTICAL-3

Part A — Practice Programs (Hour 1)
Work these in your lab-NN/ folder before the project: do the first one or two in the lab.

1. Distinct points (Point.java + Driver.java with main)
(a) Create class Point with private int x, y and a constructor.
(b) Override toString() to return "(x, y)".
(c) Override equals(Object) to be true when both x and y match, and hashCode() to agree (e.g. Objects.hash(x, y)).
(d) In Driver.main: build a Point[] with some repeated coordinates.
(e) Count distinct points: for each point, use equals() to check whether it already appeared earlier; count those that did not.
(f) Print "Distinct: N".
Expected output: 5 points with 2 repeats → "Distinct: 3".

Code:

// Point.java

```
public class P3_partA_1_point {
   private int x;
   private int y;

   P3_partA_1_point(int x,int y) {
      this.x=x;
      this.y=y;
   }

   public String toString() {
      return "(" + x + ", " + y + ")";
   }

   public boolean equals(Object o) {
      P3_partA_1_point p=(P3_partA_1_point)o;

      if(x==p.x && y==p.y)
         return true;

      return false;
   }

   public int hashCode() {
      return x*31+y;
   }
}
```

// driver.java

```
public class P3_partA_1_driver {
   public static void main(String[] args) {
      P3_partA_1_point[] arr={
         new P3_partA_1_point(1,2),
         new P3_partA_1_point(3,4),
         new P3_partA_1_point(1,2),
         new P3_partA_1_point(5,6),
         new P3_partA_1_point(3,4)
      };

      int count=0;

      for(int i=0;i<arr.length;i++) {
         boolean found=false;

         for(int j=0;j<i;j++) {
            if(arr[i].equals(arr[j])) {
               found=true;
               break;
            }
         }

         if(found==false)
            count++;
      }

      System.out.println("Distinct: " + count);
   }
}
```

```
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-3> javac P3_partA_1_point.java P3_partA_1_driver.java
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-3> java P3_partA_1_driver.java
Distinct: 3
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-3>
```

2. Duplicate card check (Card.java + Driver.java)
(a) Create class Card with private String rank, suit and a constructor.
(b) Override toString() to return rank + " of " + suit (e.g. "Queen of Hearts").
(c) Override equals()/hashCode() by rank + suit.
(d) In Driver.main: add cards to a Card[] one at a time; before adding each, use equals() to check it against the earlier cards.
(e) Print "Duplicate found: <card>" the first time a repeat appears.
Expected output: a second "Ace of Spades" → "Duplicate found: Ace of Spades"

Code:

//card.java

```
public class P3_partA_2_card {
   private String rank;
   private String suit;

   P3_partA_2_card(String rank,String suit) {
      this.rank=rank;
      this.suit=suit;
   }

   public String toString() {
      return rank + " of " + suit;
   }

   public boolean equals(Object o) {
      P3_partA_2_card c=(P3_partA_2_card)o;

      if(rank.equals(c.rank) && suit.equals(c.suit))
         return true;

      return false;
   }

   public int hashCode() {
      return rank.hashCode()+suit.hashCode();
   }
}
```

//driver.java

```
public class P3_partA_2_driver {
   public static void main(String[] args) {
      P3_partA_2_card[] cards={
         new P3_partA_2_card("Ace","Hearts"),
         new P3_partA_2_card("King","Spades"),
         new P3_partA_2_card("Ace","Spades"),
         new P3_partA_2_card("Queen","Hearts"),
         new P3_partA_2_card("Ace","Spades")
      };

      for(int i=0;i<cards.length;i++) {
         for(int j=0;j<i;j++) {
            if(cards[i].equals(cards[j])) {
               System.out.println("Duplicate found: " + cards[i]);
               return;
            }
         }
      }
   }
```

}

```
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA> cd PRACTICAL-3
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-3> javac P3_partA_2_card.java P3_partA_2_driver.java
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-3> java P3_partA_2_driver.java
Duplicate found: Ace of Spades
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-3>
```

Part B — Project Work (Hour 2): MiniBank

In Practical 2 you created the Account and Customer classes. Here you make their objects behave correctly — they print as readable text (toString) and are compared by account number (equals/hashCode) — and you add a nested Address class. Correct equals/hashCode is essential because Practical 12 will store accounts in collections that rely on it.

1. In Account, override the toString() method to return a readable line containing accountNumber, ownerName and balance.

2. In Account, override equals(Object o) and hashCode() so that two accounts are equal when their accountNumber values are equal.

3. In Customer, add a public static nested class named Address with String fields line, city and pincode plus getters; add an Address field to Customer and a getAddress() method.

4. In Customer, add a clone() method (implement Cloneable) that returns a copy of the customer.

5. In main, print accounts using toString(), compare two Account objects with equals(), and use instanceof to check an object's type.

Code:

```
public class P3_partB {

   static class Account {
      private String ac;
      private String name;
      private double balance;

      Account(String ac,String name,double balance) {
         this.ac=ac;
         this.name=name;
         this.balance=balance;
      }

      public String toString() {
         return ac + " " + name + " " + balance;
      }

      public boolean equals(Object o) {
         if(!(o instanceof Account))
            return false;

         Account a=(Account)o;
         return ac.equals(a.ac);
      }
```

```
    public int hashCode() {
        return ac.hashCode();
    }
}

static class Customer implements Cloneable {
    private String name;
    private String email;
    private String mobile;
    private Address addr;

    Customer(String name,String email,String mobile,Address addr) {
        this.name=name;
        this.email=email;
        this.mobile=mobile;
        this.addr=addr;
    }

    public Address getAddress() {
        return addr;
    }

    public Customer clone() {
        try {
            return (Customer)super.clone();
        }
        catch(CloneNotSupportedException e) {
            return null;
        }
    }

    static class Address {
        private String line;
        private String city;
        private String pin;

        Address(String line,String city,String pin) {
            this.line=line;
            this.city=city;
            this.pin=pin;
        }

        public String getLine() {
            return line;
        }

        public String getCity() {
            return city;
        }

        public String getPincode() {
            return pin;
        }
    }
}
```

```
    public static void main(String[] args) {
        Account a1=new Account("A101","Neel",5000);
        Account a2=new Account("A102","Selvy",7000);
        Account a3=new Account("A101","Neel",5000);
        System.out.println(a1);
        System.out.println(a2);

        System.out.println(a1.equals(a2));
        System.out.println(a1.equals(a3));

        Customer.Address addr=new Customer.Address("Sanskardham","Ankleshwar","393001");

        Customer c1=new Customer("Neel","neenpatel00@gmail.com","9726138765",addr);
        Customer c2=c1.clone();

        System.out.println(c2.getAddress().getCity());

        Object o=a1;

        if(o instanceof Account)
            System.out.println("It is an Account");
    }
}
```

```
PROBLEMS 7  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA> cd "c:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-3\" ; if ($?) { javac P3_partB.ja
va } ; if ($?) { java P3_partB }
A101 Neel 5000.0
A102 Selvy 7000.0
false
true
Ankleshwar
It is an Account
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-3>
```

KEY QUESTION:

1.Why must equals() and hashCode() be overridden together?
=>Because if two objects are equal using equals(), they must return the same hashCode() so collections like HashMap and HashSet work correctly.

2.What does the default Object.toString() return, and why override it?
=>The default Object.toString() returns the class name followed by the object's hash code; we override it to display meaningful and readable object information.

3.What is the difference between a static nested class and an inner class?
=>A static nested class does not need an object of the outer class, while an inner class needs an outer class object and can access its instance members.

PRACTICAL-4

Part A — Practice Programs (Hour 1)
Work these in your lab-NN/ folder before the project: do the first one or two in the lab.

1. Password strength (PasswordChecker.java + Driver.java)
(a) In PasswordChecker, check four rules — length ≥ 8, contains an uppercase letter, contains a digit, contains a special character — using regex (e.g. ".*[A-Z].*") or String methods.
(b) Write strength(String pw) that counts how many rules pass and returns "Weak" (0–1), "Medium" (2–3) or "Strong" (4).
(c) In Driver.main: test several passwords; for each, print which rules passed and the final label.
Expected output: "abc" → Weak; "Abcd1234!" → Strong.

Code:

// PasswordChecker.java

```
public class P4_partA_1_PasswordChecker {
public static String strength(String pw) {
int count=0;

boolean length=pw.length()>=8;
boolean upper=pw.matches(".*[A-Z].*");
boolean digit=pw.matches(".*[0-9].*");
boolean special=pw.matches(".*[^a-zA-Z0-9].*");

if(length) count++;
if(upper) count++;
if(digit) count++;
if(special) count++;

System.out.println("Password: " + pw);
System.out.println("Length >= 8: " + length);
System.out.println("Uppercase: " + upper);
System.out.println("Digit: " + digit);
System.out.println("Special character: " + special);

if(count<=1)
return "Weak";
else if(count<=3)
return "Medium";
else
return "Strong";
}
}
```

// driver.java

```
public class P4_partA_1_driver {
public static void main(String[] args) {
String[] passwords={
"abc",
"abcdefghi",
"Abcd1234",
"Abcd1234!"
};

for(String pw:passwords) {
String result=P4_partA_1_PasswordChecker.strength(pw);
System.out.println("Strength: " + result);
System.out.println();
}
}
}
```

```
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA> cd PRACTICAL-4
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-4> javac P4_partA_1_PasswordChecker.java P4_partA_1_driver.java
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-4> java P4_partA_1_driver.java
Password: abc
Length >= 8: false
Uppercase: false
Digit: false
Special character: false
Strength: Weak

Password: abcdefghi
Length >= 8: true
Uppercase: false
Digit: false
Special character: false
Strength: Weak

Password: Abcd1234
Length >= 8: true
Uppercase: true
Digit: true
Special character: false
Strength: Medium

Password: Abcd1234!
Length >= 8: true
Uppercase: true
Digit: true
Special character: true
Strength: Strong

PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-4>
```

2. Chat-log filter (ChatFilter.java + Driver.java)
(a) In Driver.main, define a String[] of log lines like "10:05 alice Hello there" (include one malformed line with fewer than 3 parts).
(b) Read a keyword from the user.
(c) For each line, split into time/user/message with split(" ", 3); skip lines that do not have 3 parts.
(d) If the message contains the keyword (case-insensitive), increment a counter and append "time user: message" to a StringBuilder.
(e) Print "Matches: N" and the built report.
Expected output: keyword "hello" over 3 lines (1 malformed) → "Matches: 1" and the matching line.

Code:

//ChatFilter.java

```
public class P4_partA_2_chatfilter {
public static void filter(String[] logs,String key) {
int count=0;
StringBuilder report=new StringBuilder();

for(String line:logs) {
String[] p=line.split(" ",3);

if(p.length<3)
continue;

String time=p[0];
String user=p[1];
String msg=p[2];

if(msg.toLowerCase().contains(key.toLowerCase())) {
count++;
report.append(time+" "+user+": "+msg+"\n");
}
}

System.out.println("Matches: "+count);
System.out.print(report);
}
}
```

//driver.java

```
import java.util.Scanner;

public class P4_partA_2_driver {
public static void main(String[] args) {
Scanner sc=new Scanner(System.in);

String[] logs={
"10:05 Neel Hello..!! there..",
"10:10 Kavy How are you?",
"10:15",
"10:20 Neel Good morning"
};

System.out.print("Enter keyword: ");
String key=sc.nextLine();

P4_partA_2_chatfilter.filter(logs,key);
}
}
```

```
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-4> javac P4_partA_2_chatfilter.java P4_partA_2_driver.java
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-4>
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-4> java P4_partA_2_driver.java
Enter keyword: Hello
Matches: 1
10:05 Neel: Hello..!! there..
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-4>
```

Part B — Project Work (Hour 2): MiniBank

Your classes exist, but the bank must never trust raw user input. In this lab you build MiniBank's validation and command-parsing layer — regex checks for mobile, email, PAN and IFSC, and a parser that turns a typed line such as "DEPOSIT AC0001 500" into a Command object. Practical 14 will run every menu action through this layer.

1. Create a class named Validator with public static methods that use compiled regular expressions (Pattern): isValidMobile(String), isValidEmail(String), isValidPan(String) and isValidIfsc(String); each returns a boolean.

2. Define an enum named TransactionType with the constants DEPOSIT, WITHDRAW and TRANSFER. Then define a record named Command with three fields: a TransactionType type, a String accountNumber and a long amount.

3. Create a class named CommandParser with a public static method parse(String line) that splits a line such as "DEPOSIT AC0001 500" with split(), converts the parts, and returns a Command object.

4. Create a class named StatementFormatter with a public static method buildStatement(Account account) that uses a StringBuilder to assemble a multi-line account statement string.

5. In main, test each validator with one correct and one wrong input, then parse a sample command and print its three parts.

Code:

```
import java.util.regex.Pattern;

enum TransactionType
{
DEPOSIT,
WITHDRAW,
TRANSFER
}

record Command(TransactionType type,String accountNumber,long amount){}

public class P4_partB {

static class Validator
{
static Pattern mobile=Pattern.compile("^[6-9][0-9]{9}$");
static Pattern email=Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");
static Pattern pan=Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");
static Pattern ifsc=Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");

public static boolean isValidMobile(String s)
{
return mobile.matcher(s).matches();
}

public static boolean isValidEmail(String s)
{
return email.matcher(s).matches();
}

public static boolean isValidPan(String s)
{
return pan.matcher(s).matches();
}
```

```
public static boolean isValidIfsc(String s)
{
return ifsc.matcher(s).matches();
}
}


static class CommandParser
{
public static Command parse(String line)
{
String[] a=line.split(" ");

TransactionType type=TransactionType.valueOf(a[0]);
String acc=a[1];
long amount=Long.parseLong(a[2]);

return new Command(type,acc,amount);
}
}

static class Account
{
String acc;
String name;
long balance;

Account(String acc,String name,long balance)
{
this.acc=acc;
this.name=name;
this.balance=balance;
}
}

static class StatementFormatter
{
public static String buildStatement(Account ac)
{
StringBuilder s=new StringBuilder();

s.append("Account Statement\n");
s.append("Account Number: "+ac.acc+"\n");
s.append("Name: "+ac.name+"\n");
s.append("Balance: "+ac.balance+"\n");

return s.toString();
}
}

public static void main(String[] args)
{

System.out.println("Mobile:");
System.out.println(Validator.isValidMobile("9726137865"));
System.out.println(Validator.isValidMobile("12345"));

System.out.println("Email:");
System.out.println(Validator.isValidEmail("neelpatel00@gmail.com"));
System.out.println(Validator.isValidEmail("abc@"));
```

```
System.out.println("PAN:");
System.out.println(Validator.isValidPan("VYNEE2405S"));
System.out.println(Validator.isValidPan("PQR303"));

System.out.println("IFSC:");
System.out.println(Validator.isValidIfsc("UBIN0204671"));
System.out.println(Validator.isValidIfsc("UBIN543"));

Command c=CommandParser.parse("DEPOSIT AC0001 500");

System.out.println("\nCommand:");
System.out.println("Type: "+c.type());
System.out.println("Account: "+c.accountNumber());
System.out.println("Amount: "+c.amount());

Account ac=new Account("AC001","Neel",20000);

System.out.println();
System.out.print(StatementFormatter.buildStatement(ac));
}
}
```

```
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-4> cd "c:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-4\" ; if ($?) { java
 P4_partB.java } ; if ($?) { java P4_partB }
Mobile:
true
false
Email:
true
false
PAN:
true
false
IFSC:
true
false

Command:
Type: DEPOSIT
Account: AC0001
Amount: 500

Account Statement
Account Number: AC001
Name: Neel
Balance: 20000
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-4>
```

KEY QUESTION:

1.What is the difference between String, StringBuilder and StringBuffer, and when is eachpreferred?

=>String is immutable, StringBuilder is mutable and faster for frequent changes, and StringBuffer is mutable and thread-safe; use String for fixed text, StringBuilder for single-threaded modifications, and StringBuffer for multi-threaded modifications.

2.What does the regular expression [6-9][0-9]{9} match, and why use anchors?

=>[6-9][0-9]{9} matches a 10-digit mobile number starting with 6, 7, 8, or 9. Anchors ^ and $ are used to ensure the entire input matches the pattern, not just part of it.

3.Why should input be validated at the boundary before it is used?

=>Input should be validated at the boundary to reject invalid or unsafe data early, ensuring only correct data enters the system and preventing errors later.

PRACTICAL-5

Part A — Practice Programs (Hour 1)
Work these in your lab-NN/ folder before the project: do the first one or two in the lab.

1.Shape areas: an abstract Shape with abstract area(); subclasses Circle, Rectangle,Triangle. Put a mix in a Shape[] and loop once to print each area and the running total,tracking the largest — one loop handling every shape through polymorphism.

Code:

```
abstract class Shape{
   public abstract double area();
}

class Circle extends Shape{
   private double radius;

   public Circle(double radius){
      this.radius = radius;
   }

   public double area(){
      return Math.PI * radius * radius;
   }
}

class Rectangle extends Shape{
   private double length;
   private double width;

   public Rectangle(double length, double width){
      this.length=length;
      this.width=width;
   }

   public double area(){
      return length * width;
   }
}

class Triangle extends Shape{
   private double base;
   private double height;

   public Triangle(double base, double height){
      this.base=base;
      this.height=height;
   }

   public double area(){
      return 0.5 * base * height;
   }
}
```



```
public class P5_partA_1{
```

```
    public static void main(String [] args){
        Shape[] shapes = {

            new Circle(10),
            new Rectangle(15, 25),
            new Triangle(36, 60)
        };

        double total=0;
        double largest=0;

        for(Shape shape : shapes){
            double area=shape.area();
            total+=area;

        if(area>largest){
            largest=area;
        }

        System.out.printf("Area is: %.2f\n", total);
        }
        System.out.printf("Area is: %.2f\n" , total);
        System.out.printf("Largest area is: %.2f\n" , largest);
    }
}
```

```
        at P5_partA_1.main(P5_partA_1.java:65)
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-5> cd "c:\Users\neenu.NEEL\OneDriv
ava } ; if ($?) { java P5_partA_1 }
Area is: 314.16
Area is: 689.16
Area is: 1769.16
Area is: 1769.16
Largest area is: 1080.00
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-5>
```

2. Payroll: an abstract Employee with abstract monthlySalary(); FullTime (fixed), PartTime (hours × rate) and Intern (stipend), each calling super for the shared name/id. Loop a mixed array to print each salary and the total; use instanceof to add a note only for interns.

Code:

```
abstract class Employee{
    protected String name;
    protected int id;

    public Employee(String name,int id){
        this.name=name;
        this.id=id;
    }

    public abstract double monthlySalary();
}
```



```
class FullTime extends Employee{
```

```
    private double salary;

    public FullTime(String name,int id,double salary){
        super(name,id);
        this.salary=salary;
    }

    public double monthlySalary(){
        return salary;
    }
}

class PartTime extends Employee{
    private double hours;
    private double rate;

    public PartTime(String name,int id,double hours,double rate){
        super(name,id);
        this.hours=hours;
        this.rate=rate;
    }

    public double monthlySalary(){
        return hours*rate;
    }
}

class Intern extends Employee{
    private double stipend;

    public Intern(String name,int id,double stipend){
        super(name,id);
        this.stipend=stipend;
    }

    public double monthlySalary(){
        return stipend;
    }
}

public class P5_partA_2{
    public static void main(String[] args){
        Employee[] employees={
            new FullTime("Neel",101,5000),
            new PartTime("Selvy",102,80,300),
            new Intern("Kavy",103,15000),
        };

        double total=0;

        for(Employee employee:employees){
            double salary=employee.monthlySalary();
            total+=salary;

            System.out.printf("%s Salary: %.2f\n",employee.name,salary);
```

```
        if(employee instanceof Intern){
            System.out.println("Note: This employee is an Intern.");
        }
    }

    System.out.printf("Total Payroll: %.2f\n",total);
  }
}
```

```
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-5> cd "c:\Use
ava } ; if ($?) { java P5_partA_2 }
Neel Salary: 50000.00
Selvy Salary: 24000.00
Kavy Salary: 15000.00
Note: This employee is an Intern.
Total Payroll: 89000.00
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-5>
```

Part B — Project Work (Hour 2): MiniBank

A real bank has different account types. Building on the single Account class, this lab makes Account abstract and adds three subclasses — SavingsAccount, CurrentAccount and FixedDepositAccount — each with its own interest and withdrawal rules chosen automatically by polymorphism. This lets the rest of MiniBank treat every account through one common type.

1. Make Account an abstract class. Add two abstract methods: double interestRate() and boolean canWithdraw(long amount).

2. Create a class SavingsAccount that extends Account with a long field minBalance; interestRate() returns 4.0; canWithdraw() returns true only if the balance stays greater than or equal to minBalance.

3. Create a class CurrentAccount that extends Account with a long field overdraftLimit; interestRate() returns 0; canWithdraw() allows the balance to fall to minus overdraftLimit.

4. Create a class FixedDepositAccount that extends Account; interestRate() returns 7.0; canWithdraw() returns false (the deposit is locked).

5. Call super(...) from each subclass constructor to initialise the inherited fields.

6. In main, place objects of the three types in an Account[] array, loop through it and call interestRate() on each (this demonstrates run-time polymorphism); use a pattern instanceof check to handle one type specially.

Code:

```
abstract class Account{
   protected String name;
   protected long accountNumber;
   protected long balance;

   public Account(String name,long accountNumber,long balance){
      this.name=name;
      this.accountNumber=accountNumber;
      this.balance=balance;
   }

   public abstract double interestRate();

   public abstract boolean canWithdraw(long amount);
}

class SavingsAccount extends Account{
   private long minBalance;

   public SavingsAccount(String name,long accountNumber,long balance,long minBalance){
      super(name,accountNumber,balance);
      this.minBalance=minBalance;
   }

   public double interestRate(){
      return 4.0;
   }

   public boolean canWithdraw(long amount){
      return balance-amount>=minBalance;
   }
}

class CurrentAccount extends Account{
   private long overdraftLimit;

   public CurrentAccount(String name,long accountNumber,long balance,long overdraftLimit){
      super(name,accountNumber,balance);
      this.overdraftLimit=overdraftLimit;
   }

   public double interestRate(){
      return 0.0;
   }

   public boolean canWithdraw(long amount){
      return balance-amount>=-overdraftLimit;
   }
}

class FixedDepositAccount extends Account{

   public FixedDepositAccount(String name,long accountNumber,long balance){
      super(name,accountNumber,balance);
   }
```

```
        return 7.0;
    }

    public boolean canWithdraw(long amount){
        return false;
    }
}

public class P5_partB{
    public static void main(String[] args){
        Account[] accounts={
            new SavingsAccount("Neel",101,5000,100),
            new CurrentAccount("Selvy",102,6000,1500),
            new FixedDepositAccount("Kavy",103,3000)
        };

        for(Account account:accounts){
            System.out.println("Account Holder: "+account.name);
            System.out.println("Interest Rate: "+account.interestRate()+"%");

            if(account instanceof FixedDepositAccount){
                System.out.println("Note: Fixed Deposit is locked.");
            }

            System.out.println();
        }
    }
}
```

```
PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-5> cd "c:\U
a } ; if ($?) { java P5_partB }
Account Holder: Neel
Interest Rate: 4.0%

Account Holder: Selvy
Interest Rate: 0.0%

Account Holder: Kavy
Interest Rate: 7.0%
Note: Fixed Deposit is locked.

PS C:\Users\neenu.NEEL\OneDrive\Desktop\Practical\JAVA\PRACTICAL-5>
```

KEY QUESTION:

1.What is the difference between an abstract class and a concrete class?

=>An abstract class cannot be instantiated and may contain abstract methods, while a concrete class can be instantiated and has implemented methods.

2.What is dynamic method dispatch (run-time polymorphism)?

=>Dynamic method dispatch is when Java decides at run time which overridden method to call based on the actual object.

3.What does the super keyword do in a subclass constructor?

=>The super keyword calls the parent class constructor to initialize the inherited fields and properties.